

PriceWatch - App Flutter

Explicacao completa do aplicativo, telas, estado global, models e integracao com API

Material de estudo para apresentacao: explica a arquitetura, os fluxos, os arquivos principais, as escolhas tecnicas e o papel de cada parte do codigo.

Projeto: PriceWatch - comparador e monitorador de precos

1. Visao geral do app

O aplicativo Flutter e a interface usada pelo usuario final. Ele permite criar conta, entrar, buscar produtos, importar resultados vindos da API, criar alertas de preco, visualizar produtos monitorados, consultar notificacoes, ver detalhes do produto e acessar historicos locais de busca e compra.

A escolha principal foi separar responsabilidades: telas cuidam da interface, services cuidam da comunicacao HTTP, models transformam JSON em objetos Dart, core guarda configuracoes globais, e components reaproveitam widgets visuais.

Fluxo principal do usuario

Login/Register

- > AppSession recebe usuario e token
- > Home lista alertas reais da API
- > Buscar produto consulta /products/search
- > Se vazio, importa de /external-products/import
- > Usuario define preco alvo
- > POST /alerts cria monitoramento
- > Notifications consome /notifications

2. Dependencias e configuracao

Arquivo	Codigo/Dependencia	Por que existe
pubspec.yaml	http	Permite fazer chamadas HTTP para a API Express.
pubspec.yaml	provider	Disponibiliza AppSession como estado global para o app inteiro.
pubspec.yaml	flutter_dotenv	Le API_BASE_URL do arquivo .env sem fixar a URL no codigo.
pubspec.yaml	image_picker	Permite escolher imagem de perfil pela galeria.
pubspec.yaml	assets: .env	Inclui o arquivo .env nos assets carregados pelo Flutter.

Ponto para explicar: API_BASE_URL nao e segredo. E apenas o endereco da API. Em Android Emulator, normalmente deve ser `http://10.0.2.2:3000`; no navegador local pode ser `http://localhost:3000`; em celular fisico deve ser o IP da maquina na rede.

3. main.dart - entrada do aplicativo

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await dotenv.load(fileName: '.env');

  runApp(
    ChangeNotifierProvider.value(
      value: AppSession.instance,
      child: const PriceComparatorApp(),
    ),
  );
}
```

- `WidgetsFlutterBinding.ensureInitialized()` garante que o Flutter esteja pronto antes de carregar assets como `.env`.
- `dotenv.load` carrega a URL da API antes do primeiro widget ser exibido.
- `ChangeNotifierProvider.value` expõe `AppSession.instance` para a arvore de widgets.
- `PriceComparatorApp` monta `MaterialApp`, tema, rota inicial e tabela de rotas.
- `initialRoute`: `AppRoutes.login` faz o app abrir na tela de autenticacao.

`MaterialApp` concentra configuracoes globais: titulo, remocao da faixa de debug, tema Material 3, cor de fundo e mapa de rotas. Cada rota aponta para uma tela, por exemplo `/home` para `HomeScreen` e `/notifications` para `NotificationsScreen`.

4. Pasta core - configuracoes compartilhadas

Arquivo	Responsabilidade	Como explicar
api_config.dart	Define a baseUrl da API.	Primeiro tenta --dart-define, depois .env, e por ultimo localhost. Isso deixa o app flexível para web, emulador e producao.
app_session.dart	Guarda estado global de usuario, token, historicos e imagem.	Extends ChangeNotifier para avisar telas quando dados mudam.
app_routes.dart	Centraliza nomes das rotas.	Evita strings espalhadas, como '/home' repetido em muitos lugares.
app_colors.dart	Paleta de cores.	Padroniza identidade visual e evita cores soltas no codigo.
app_text_styles.dart	Estilos de texto.	Mantem tipografia consistente em cards, titulos e legendas.

AppSession explicado por partes

```
class AppSession extends ChangeNotifier {
  static final AppSession instance = AppSession._();
  String name = '';
  String email = '';
  String? token;
  String? userId;
  Uint8List? profileImageBytes;
}
```

- O construtor privado AppSession._() impede criar varias sessoes por engano.
- instance cria uma instancia unica, funcionando como sessao global.
- token guarda o JWT recebido no login e usado em rotas autenticadas.
- purchaseHistory e searchHistory guardam historicos locais do usuario.
- notifyListeners() e chamado apos mudancas para atualizar widgets que observam a sessao.

Metodos importantes: setAuthenticatedUser salva userId, nome, email e token apos login/cadastro; updateProfile atualiza dados do perfil; clear limpa a sessao no logout; addToSearchHistory e addToPurchaseHistory evitam duplicacao e colocam itens recentes no inicio.

5. Camada de comunicacao com API

A camada services foi separada em duas partes: ApiClient e ApiService. ApiClient sabe como fazer HTTP generico; ApiService sabe quais endpoints existem no backend.

ApiClient

```
Uri _uri(String path, [Map<String, String?> query = const {}]) {
  return Uri.parse('${ApiConfig.baseUrl}$path').replace(
    queryParams: filteredQuery.isEmpty ? null : filteredQuery,
  );
}
```

- _uri monta a URL completa combinando API_BASE_URL com o caminho, como /alerts.
- filteredQuery remove parametros nulos para nao enviar query vazia.
- _headers adiciona Content-Type, Accept e Authorization quando authenticated=true.
- get, post, put e delete encapsulam os metodos HTTP.
- _decode converte JSON e transforma erro HTTP em ApiException.

ApiService

Metodo	Endpoint	Papel no app
login	POST /auth/login	Entra na conta e salva token em AppSession.
register	POST /auth/register	Cria usuario e tambem autentica.
searchProducts	GET /products/search	Busca produtos ja existentes no banco.

Metodo	Endpoint	Papel no app
trendingProducts	GET /products/trending	Lista produtos recentes/em destaque.
importExternalProducts	POST /external-products/import	Pede para a API buscar dados externos e salvar no banco.
listAlerts	GET /alerts	Carrega produtos monitorados do usuario logado.
createAlert	POST /alerts	Cria alerta com productId e targetPrice.
updateAlert	PUT /alerts/:id	Altera preco alvo ou estado do alerta.
deleteAlert	DELETE /alerts/:id	Remove alerta monitorado.
listNotifications	GET /notifications	Carrega notificacoes reais derivadas dos alertas.
findOffers	GET /products/:id/offers	Mostra lojas/ofertas do mesmo produto.
findPriceHistory	GET /products/:id/price-history	Monta historico/grafico de precos.

6. Models - objetos usados pelas telas

Models convertem dados JSON da API para objetos Dart. Isso evita que as telas fiquem acessando `json['campo']` diretamente, o que seria mais fragil e espalharia regra de conversao por todo o app.

Model	Campos principais	Logica importante
ProductModel	id, alertId, name, store, category, currentPrice, targetPrice, previousPrice, emoji	fromJson converte produto comum; fromAlertJson adapta resposta de /alerts; priceChangePercent calcula variacao; targetProgress calcula progresso ate o alvo; isTargetReached verifica se preco chegou no alvo.
UserModel	id, name, email	fromJson converte usuario retornado pela API; initials gera iniciais para avatar.
NotificationModel	id, title, description, timeAgo, type, isRead, dateGroup	fromJson converte notificacoes reais; _parseType transforma strings da API em enum Dart.
ApiException	message, statusCode	Permite exibir mensagem de erro e tratar casos como 409, alerta duplicado.

7. Telas de autenticacao

AuthScreen alterna entre LoginForm e RegisterForm usando `_isLogin`. O botao de alternancia muda o estado local com `setState`. Quando o formulario termina com sucesso, `onSubmit` navega para HomeScreen.

LoginForm

- Usa `TextEditingController` para ler e-mail e senha.
- Valida campos vazios antes de chamar API.
- `_isLoading` bloqueia duplo clique no botao.
- `ApiService.instance.login` faz `POST /auth/login`.
- Se der `ApiException`, mostra `error.message`; se a API estiver fora, mostra erro de conexao.

RegisterForm

- Valida nome, e-mail, senha, confirmacao e aceite dos termos.
- Usa `RegExp` para conferir formato basico de e-mail.
- Confere se senha e confirmacao sao iguais.
- `ApiService.instance.register` envia nome, email e senha para a API.
- Ao receber token, `AppSession` e preenchido pelo `ApiService`.

8. HomeScreen - tela principal dos alertas

```
Future<void> _loadAlerts() async {
  final products = await ApiService.instance.listAlerts();
  if (!mounted) return;
  setState(() => _products = products);
}
```

- `_products` guarda os alertas em forma de `ProductModel`.
- `_isLoading` controla indicador de carregamento.
- `_error` guarda mensagem quando a API falha.
- `_totalMonitoring` conta produtos monitorados.
- `_activeAlerts` conta alertas cujo alvo ainda não foi atingido.
- `_totalSaved` calcula economia comparando `previousPrice` e `currentPrice`.
- `FloatingActionButton` abre `SearchScreen` para adicionar novo alerta.
- `_deleteAlert` chama `DELETE /alerts/:id` e remove o item da lista local.

Escolha técnica: a `Home` não cria dados falsos. Ela depende de `listAlerts`, portanto reflete o banco MySQL. Quando volta de telas como busca, catálogo ou edição, ela chama `_reloadProducts` para atualizar a lista.

9. Busca, catálogo e importação externa

`SearchScreen` e `CatalogScreen` usam `ProductModel` e `ApiService` para mostrar produtos. A busca tem `debounce` com `Timer`: o app espera alguns milissegundos depois da digitação antes de chamar a API, evitando muitas requisições enquanto o usuário digita.

```
if (products.isEmpty) {
  await ApiService.instance.importExternalProducts(query: query);
  products = await ApiService.instance.searchProducts(query: query);
}
```

- Primeiro o app consulta produtos já salvos no banco.
- Se não encontrar, pede para a API importar produtos externos.
- Depois consulta novamente o banco, agora alimentado com produtos importados.
- Ao tocar em um produto, o app navega para `NewAlertScreen` com o produto como argumento.

10. NewAlertScreen - criação de monitoramento

- Recebe `ProductModel` por `ModalRoute.of(context)!.settings.arguments`.
- Mostra produto, loja, categoria e preço atual.
- Usuário digita `targetPrice`, ou seja, preço alvo.
- `_submit` valida se o valor é numérico, positivo e menor que preço atual.
- `ApiService.createAlert` envia `productId` e `targetPrice`.
- Se a API retornar 409, mostra que o produto já está sendo monitorado.
- No sucesso, mostra `SnackBar` e volta para `Home` removendo rotas anteriores.

Por que `targetPrice` precisa ser menor que `currentPrice`? Porque o objetivo do app é avisar quando o preço cair. Se o alvo for maior ou igual ao preço atual, o alerta já estaria atingido ou não faria sentido para monitoramento de queda.

11. ProductDetailScreen, ofertas e histórico

- A tela de detalhes apresenta informações completas de um produto selecionado.
- Chama `findOffers` para listar ofertas do mesmo produto em lojas diferentes.
- Chama `findPriceHistory` para mostrar evolução do preço.
- Usa dados derivados como `isCheapest`, `previousPrice` e `variationPercentage` para comunicar se o preço caiu ou subiu.

- Também permite criar ou acompanhar alerta dependendo do fluxo que levou o usuário até a tela.

12. NotificationsScreen - notificações reais

```
Future<void> _loadNotifications() async {
  final notifications = await ApiService.instance.listNotifications();
  if (!mounted) return;
  setState(() => _notifications = notifications);
}
```

- Antes a tela usava mock local; agora consome GET /notifications.
- _isLoading mostra CircularProgressIndicator.
- _error mostra mensagem e botão Tentar novamente.
- _selectedFilter filtra por Todas, Queda de preço, Alertas e Sistema.
- _markAllAsRead altera isRead localmente para simular leitura.
- _removeNotification remove item da lista local ao arrastar o card.
- As notificações são agrupadas por dateGroup, como HOJE e ONTEM.

13. ProfileScreen e históricos locais

- Mostra nome, e-mail e iniciais vindas de AppSession.
- Permite escolher foto com image_picker e salva bytes em AppSession.
- Mostra telas auxiliares: compras, histórico de busca, sobre, contato, termos e senha.
- As preferências de notificação são estados locais de interface, ainda sem persistência na API.
- Logout limpa dados de sessão e volta para login.

14. Components - widgets reutilizáveis

Componente	Uso
PrimaryButton	Botão principal padronizado com texto, ícone e ação.
CustomTextField	Campo reutilizável com label, ícone, erro e opção de senha.
ProductCard	Card dos produtos monitorados na Home.
NotificationCard e SwipeableNotificationCard	Visualização e remoção por gesto das notificações.
BottomNavBar	Navegação inferior entre Home, Catálogo, Notificações e Perfil.
StatCard	Resumo numérico no topo da Home.
PriceProgressBar e PriceInfoCard	Indicadores visuais de preço e progresso até o alvo.
ReusableText e AppText	Encapsulam textos com estilo padronizado.

15. Como defender as escolhas técnicas

- Arquitetura por camadas: UI em screens/components, estado em core/app_session.dart, integração em services, dados em models.
- Provider foi usado porque atende ao requisito de estado global e é simples para um app acadêmico.
- http foi usado porque o app precisa consumir uma API externa/própria.
- .env evita URL fixa no código e facilita trocar entre navegador, emulador e celular.
- A API é a fonte da verdade para produtos e alertas; históricos locais são apenas experiência de usuário.
- Loading, erro e lista vazia aparecem nas telas principais, cumprindo tratamento mínimo dos estados.

16. Roteiro rapido para apresentacao

- Comece dizendo que o app e um comparador/monitorador de precos feito em Flutter.
- Explique que o usuario faz login e recebe token JWT da API.
- Mostre que AppSession guarda token e dados globais.
- Mostre que ApiClient adiciona Authorization automaticamente quando a rota exige login.
- Demonstre a busca: primeiro banco, depois importacao externa se nao houver resultado.
- Demonstre criacao de alerta e explique productId + targetPrice.
- Abra notificacoes e explique que agora elas vêm da API, derivadas dos alertas.
- Feche com a arquitetura: telas, componentes, models, services e core.

17. Guia arquivo por arquivo - Flutter

Esta seção é para estudo antes da apresentação. Ela resume o que cada arquivo relevante faz e qual frase usar ao explicar.

Arquivo	Explicação detalhada
lib/main.dart	Ponto de entrada. Inicializa Flutter, carrega .env, registra Provider global e cria MaterialApp com tema e rotas.
lib/core/api_config.dart	Resolve a URL da API. A ordem é: dart-define, .env e fallback localhost. Isso evita trocar código quando muda ambiente.
lib/core/app_session.dart	Sessão global observável. Guarda usuário, token, imagem de perfil e históricos. notifyListeners avisa as telas quando algo muda.
lib/core/app_routes.dart	Lista centralizada de nomes de rotas. Ajuda a evitar erros de digitação em Navigator.pushNamed.
lib/core/app_colors.dart	Paleta do sistema. Separar cores facilita manter identidade visual e alterar tema depois.
lib/core/app_text_styles.dart	Tipografia padronizada. Evita que cada tela invente tamanho/peso de texto diferente.
lib/services/api_client.dart	Cliente HTTP genérico. Monta URI, headers, Authorization, executa get/post/put/delete e decodifica respostas.
lib/services/api_service.dart	Camada de casos de uso HTTP. Cada método representa uma ação do app: login, buscar produto, criar alerta, listar notificações.
lib/services/api_exception.dart	Classe de erro usada para carregar mensagem e statusCode da API até a UI.
lib/models/product_model.dart	Objeto principal do app. Além de campos, possui factories e cálculos como variação, progresso e alvo atingido.
lib/models/user_model.dart	Representa usuário retornado pela API. Usado na sessão e na exibição de perfil.
lib/models/notification_model.dart	Representa notificação real da API. Converte string de tipo para enum Dart.

18. Guia das telas - Flutter

Tela	Responsabilidade e pontos de fala
AuthScreen	Container visual de login/cadastro. Alterna formulários com _isLogin e navega para Home após sucesso.
LoginForm	Controla campos de email/senha, validação local, loading e chamada ApiService.login.
RegisterForm	Valida nome, email, senha, confirmação e termos. Chama ApiService.register.
HomeScreen	Tela principal. Lista alertas reais, calcula estatísticas, permite remover/editar/acessar produto e navegar.
SearchScreen	Busca produtos com debounce. Se a API não retorna nada, solicita importação externa e busca novamente.
CatalogScreen	Lista catálogo de produtos do banco com filtro e busca; registra histórico local ao abrir detalhe.
NewAlertScreen	Recebe produto por argumento, valida preço alvo e cria alerta por POST /alerts.
EditAlertScreen	Recebe produto monitorado, valida novo targetPrice e chama PUT /alerts/:id.
AllAlertsScreen	Mostra todos os alertas quando Home limita a quantidade exibida.
ProductDetailScreen	Exibe produto, ofertas de lojas, histórico e informações de preço.
NotificationsScreen	Carrega GET /notifications, aplica filtros, agrupa por data e permite leitura/remoção local.
ProfileScreen	Mostra usuário da AppSession, foto, históricos, preferências visuais e logout.
PurchasesScreen	Exibe purchaseHistory local: produtos que o usuário acessou.
SearchHistoryScreen	Exibe searchHistory local: produtos consultados.
About/Contact/Terms/Password/EditProfile	Telas auxiliares de conta e informação; completam o fluxo de perfil.

19. Guia dos componentes - Flutter

Componente	Por que existe
AuthToggleButton	Permite alternar visualmente entre entrar e cadastrar.
BottomNavBar	Centraliza navegacao inferior para nao repetir codigo em cada tela.
CustomTextField	Campo padrao com label, icone, controller, erro e opcao de senha.
PrimaryButton	Botao de acao principal reutilizavel, com loading textual nas telas.
ProductCard	Card de alerta/produto monitorado com acoes de editar, remover, acessar e abrir detalhes.
NotificationCard	Desenha uma notificacao com icone, titulo, descricao, horario e estado lido/nao lido.
SwipeableNotificationCard	Adiciona gesto de arrastar para excluir notificacao da lista local.
DeleteIconButton	Botao visual consistente para remocao.
PriceInfoCard	Mostra informacoes resumidas de preco.
PriceProgressBar	Representa progresso visual ate o preco alvo.
StoreRow	Linha de oferta/loja no detalhe do produto.
StatCard	Pequenos indicadores da Home, como monitorando e economia.
ReusableText/AppText/SectionTitle/ProductListTile/SocialButton	Componentes de texto, secoes e itens para manter UI consistente.

20. Estados de tela que o professor pode perguntar

Estado	Onde aparece	Como funciona
Carregando	Home, Notifications, Search, Catalog	_isLoading controla CircularProgressIndicator ou texto de botao.
Erro	Login, Register, Home, NewAlert, Notifications	ApiException traz message/statusCode; a tela mostra mensagem amigavel.
Vazio	Home, Search, Notifications	Quando lista vem vazia, mostra orientacao ao usuario.
Sucesso	NewAlert, Home delete	SnackBar informa acao concluida.
Duplicado	NewAlert	Status 409 vira mensagem 'Este produto ja esta sendo monitorado'.

21. Perguntas provaveis e respostas curtas

- Por que Provider? Porque e uma solucao de estado global simples e suficiente para compartilhar sessao, token e historicos.
- Por que .env? Para trocar URL da API sem alterar codigo Dart.
- Por que models? Para converter JSON em objetos seguros e evitar logica espalhada nas telas.
- Por que ApiClient e ApiService separados? Um sabe HTTP generico; o outro sabe regras/endpoints do projeto.
- O app usa dados externos? Sim, mas quem busca e importa os dados externos e a API; o Flutter consome a API propria.
- O que ainda e local? Historicos de busca/compra, leitura/remocao de notificacao e preferencias do perfil.